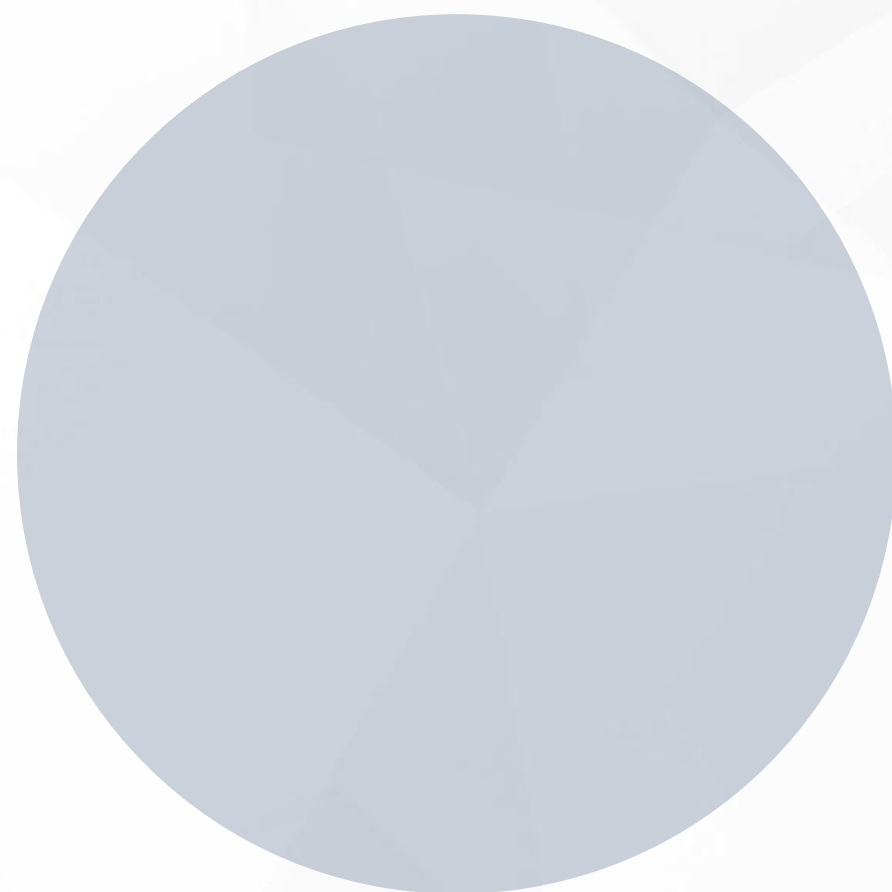




KEMENTERIAN  
KETENAGAKERJAAN  
REPUBLIK INDONESIA



Project Based  
Learning



J.620100.018.02

# MENGIMPLEMENTASIKAN PEMROGRAMAN BERORIENTASI OBJEK

**Program Pelatihan :**

**Pengembangan Web dengan Node.js dan React**

Smart **Creative Skills** Team ©

**100  
baris  
berulang**

**10  
baris yang  
bisa  
digunakan  
berulang-  
ulang**

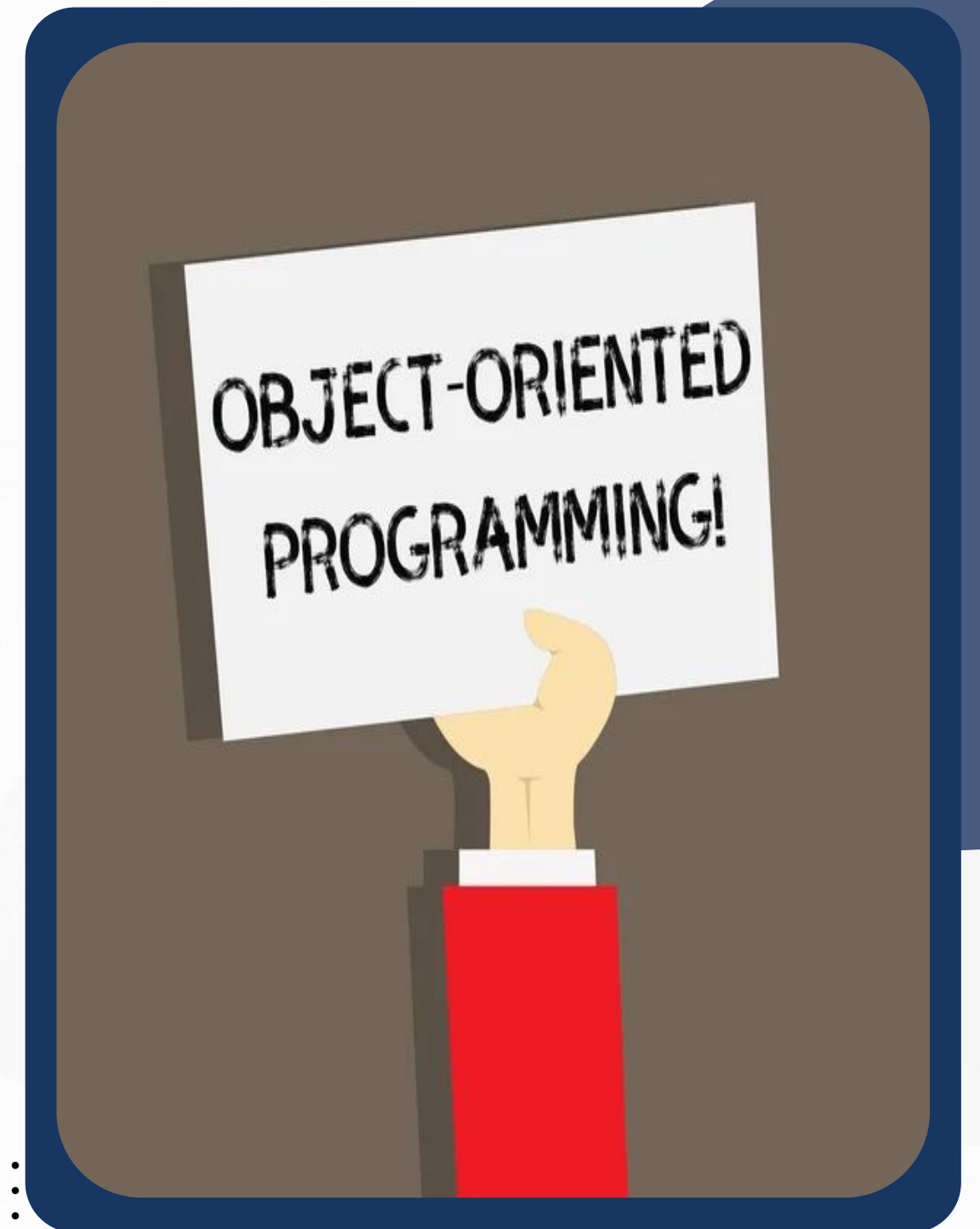


# TUJUAN

Setelah mengikuti pelatihan ini, peserta kompeten mengimplementasikan pemrograman berorientasi objek dalam proyek aplikasi web sederhana menggunakan Node.js dan React sesuai standar pemrograman berorientasi objek yang ada.

# MANFAAT

Agar peserta terbiasa berpikir logis melalui pendekatan pemrograman berbasis objek, serta berpengalaman langsung menggunakan Node.js dan React sehingga siap menghadapi kebutuhan dunia kerja maupun mengembangkan website secara mandiri



# ROAD MAP PELATIHAN

Tranning Need Analysis (TNA)

Pengembangan Web dengan  
Node.js dan React

**Mengimplementasikan  
pemrograman berorientasi objek**

01

02

03

# SAFETY INDUCTION

Membangun budaya keselamatan dan tanggung jawab digital.

Mencegah risiko kecelakaan, kerusakan alat, dan pelanggaran data.



Keamanan Fisik Peralatan TIK



Keamanan Data dan  
Informasi



Prosedur Darurat

# Pemrograman Berorientasi Objek

# INTRODUCTION

- ▶ Konsep atau cara bangun program/aplikasi/software dengan menggunakan beberapa objek yang saling berkaitan satu dengan yang lainnya.
- ▶ Kelebihan :
  - Syntax lebih terstruktur
  - efisien dan Efektif dalam sistem dengan skala besar
  - mempermudah proses pengembangan dan pemeliharaan sistem

# KARAKTERISTIK



## Abstraction

cara melihat object dalam bentuk sederhana



## Encapsulation

mekanisme untuk memproteksi suatu proses



## Inheritance

mewariskan attribute dan method sebuah class kepada class turunannya



## Polymorphism

object melakukan tindakan yang secara prinsip sama tetapi secara proses berbeda

| ELEMEN KOMPETENSI                                                      | KRITERIA UNJUK KERJA                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Membuat program berorientasi objek dengan memanfaatkan <b>class</b> | <p>1.1 Program dengan menggunakan <b>class</b> dibuat.</p> <p>1.2 Properti <b>class</b> yang akan direalisasikan dalam bentuk prosedur/fungsi dibuat.</p> <p>1.3 Data didalam <b>class</b> dibuat mandiri.</p> <p>1.4 Hak akses dari tipe data (<b>private, protected, public</b>) dikelola.</p> |

# ABSTRACTION

# Definisi

- ▶ Class dalam PBO adalah sebuah *blueprint* atau template untuk menciptakan *object-object* yang memiliki atribut (data) dan metode (fungsi) yang terkait.
- ▶ Class dalam Javascript adalah sebuah *functions* yang spesial, dan seperti *function* yang kita ketahui sebelumnya bahwa *functions* dapat dideklarasikan dan dipanggil, begitu pula dengan *Class*

# Aturan Penamaan Class

- ▶ Nama sebuah class sebaiknya menggunakan huruf kapital pada huruf pertamanya.
- ▶ Jika terdapat dua kata atau lebih maka huruf pertama pada kata yang selanjutnya harus kapital.

```
class Car{} // BENAR  
class car{} // SALAH  
class ElectricCar{} // BENAR  
class electricCar{} // SALAH
```

# Deklarasi Class di Javascript

Membuat sebuah *Class* yaitu dengan mendeklarasikannya yaitu dengan menuliskan class diikuti dengan nama classnya.

```
class Car {  
  constructor(brand){  
    this.brand = brand;  
  }  
}
```

# Deklarasi Class dengan variabel

Membuat sebuah *Class* dengan membuat sebuah variabel.  
Class tersebut boleh diberikan nama atau tidak diberi nama.

```
var Car = class {  
    constructor(brand){  
        this.brand = brand;  
    }  
}
```

```
var Car = class Car2 {  
    constructor(brand){  
        this.brand = brand;  
    }  
}
```

# constructor() pada class Javascript

- ▶ Constructor adalah methods yang selalu dijalankan paling pertama ketika sebuah class dipanggil (*instance*).
- ▶ Constructor dapat menerima parameter yang mengirim nilai ke dalam class tersebut.
- ▶ Sintaks constructor pada class merupakan method khusus, dimana dilakukan inisialisasi properties, yang akan dieksekusi secara otomatis ketika class dibuat, dan harus memiliki nama “constructor”
- ▶ Jika tidak dituliskan constructor maka Javascript akan menambahkan method constructor kosong secara otomatis.

# this. pada class Javascript

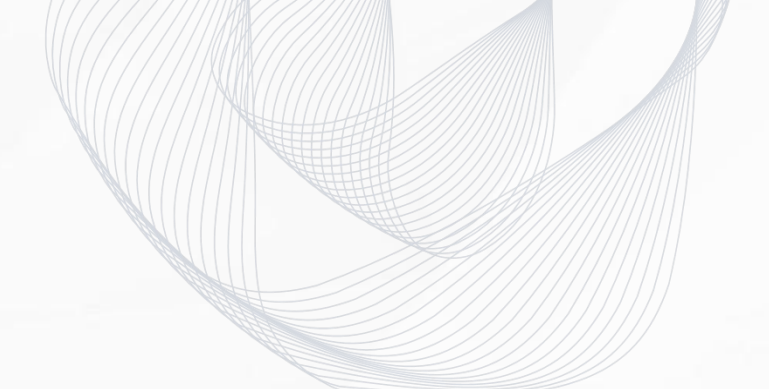
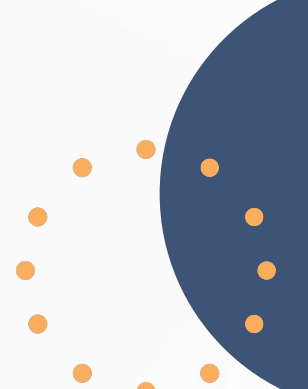
- ▶ **this.** adalah sintaks untuk menunjuk class atau seluruh body di class itu sendiri.
- ▶ Sintaks **this.** hanya hidup di dalam deklarasi class dan tidak bisa dipanggil di luar class.
- ▶ Deklarasi sebuah class cara untuk memanggil property atau methods adalah dengan menambahkan this lalu diikuti dengan titik dan nama property atau methodsnya.

# Membuat Object dari Class (Instantiate Class)

- ▶ Seperti function pada umumnya yang terdapat deklarasi dan pemanggilan function atau instance.
- ▶ Class juga memiliki cara untuk pemanggilannya

```
class Car {  
  constructor(brand){  
    this.brand = brand;  
  }  
}  
  
//Instantiate the class  
var pajero = new Car('Mitsubishi');  
console.log(pajero.brand); // Output: Mitsubishi
```

# Method

- 
- ▶ Kita juga dapat membuat method sendiri, dengan sintaks yang sudah biasa kita gunakan.
- 
- 
- 
- 

# Method

- ▶ Method adalah fungsi yang didefinisikan di dalam class.
- ▶ Method digunakan untuk memberikan perilaku (operasi) pada objek yang dibuat dari class tersebut.

```
class Car {  
  
    // method  
    sound() {  
        return "vroomm... bip..bip";  
    }  
}  
  
//Instantiate the class  
var pajero = new Car();  
console.log(pajero.sound()); // output: vroomm... bip..bip
```

# Method dengan Parameter

- ▶ Method dengan parameter adalah method (fungsi dalam class) yang menerima input dari luar saat dipanggil.
- ▶ Parameter memungkinkan method bekerja lebih fleksibel, tergantung nilai yang diberikan.

```
class Car {  
  
    // method  
    sound(x) {  
        return x + ", My car makes a sound like: vroomm... bip..bip";  
    }  
}  
  
//Instantiate the class  
var pajero = new Car();  
console.log(pajero.sound("Hello")); // Output: Hello, This car makes a sound like: vroomm... bip..bip
```

# Getters dan Setters

- ▶ getter dan setter sebagai cara yang aman dan terkontrol dalam mengakses serta mengubah nilai properti.
- ▶ Getter adalah method khusus yang digunakan untuk mengambil nilai dari properti privat atau internal dalam suatu class.
- ▶ Setter adalah method khusus yang digunakan untuk mengubah atau menetapkan nilai suatu properti.

# Getters dan Setters

- ▶ Getter (get): metode untuk mengambil/membaca nilai properti.
- ▶ Setter (set): metode untuk mengatur/mengubah nilai properti.

```
class Car{
  constructor(brand) {
    this._carname = brand;
  }
  get carname() {
    return this._carname;
  }
  set carname(x) {
    this._carname = x;
  }
}

newcar = new Car('Pajero');
newcar.carname = 'Alphard'; //mengubah nilai carname
console.log(newcar.carname); // Output: Alphard
```

# Static Method

Static methods didefinisikan hanya untuk class itu sendiri

```
class Car{
  constructor(brand) {
    this._carname = brand;
  }
  static hai() {
    return "Hai, this is a static method";
  }
}

newcar = new Car('Pajero');
console.log(Car.hai()); // Output: Hai, this is a static method
```

| ELEMEN KOMPETENSI                                                                             | KRITERIA UNJUK KERJA                                                                                                                                      |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2. Menggunakan tipe data dan <i>control program</i> pada metode atau operasi dari suatu kelas | <p>2.1 Tipe data diidentifikasi.</p> <p>2.2 Sintaks program dikuasai sesuai dengan bahasa pemrogramannya.</p> <p>2.3 <i>Control program</i> dikuasai.</p> |



# Class dengan Tipe Data dan Control Program

- ▶ Dalam class kita dapat mendeklarasikan berbagai properti (data) dan method (fungsi).
- ▶ Class juga dapat mengandung kontrol program agar objek dapat mengambil keputusan atau melakukan perulangan sesuai logika tertentu.

```
class Car {  
  constructor(kecepatan) {  
    this.kecepatan = kecepatan; // number  
  }  
  
  // Method: tampilkan informasi mobil  
  info() {  
    console.log(`Mobil memiliki kecepatan ${this.kecepatan} km/jam.`);  
  
    // Control program: if-else  
    if (this.kecepatan > 0) {  
      console.log("Mobil sedang berjalan.");  
    } else {  
      console.log("Mobil sedang berhenti.");  
    }  
  }  
}
```

# Class dengan Tipe Data dan Control Program

- ▶ Cara instantiate sama seperti pada class lainnya.

```
// Instantiate the class
var mobil1 = new Car(80);
var mobil2 = new Car(0);

// Jalankan method untuk masing-masing objek
mobil1.info(); // Output: Mobil memiliki kecepatan 80 km/jam. Mobil sedang berjalan.
mobil2.info(); // Output: Mobil memiliki kecepatan 0 km/jam. Mobil sedang berhenti.
```

# ENCAPSULATION

# Definisi

- ▶ Bertujuan untuk menyembunyikan detail internal dari objek dan hanya menyediakan antarmuka untuk berinteraksi dengannya.
- ▶ Data disimpan dalam objek (disebut private data).
- ▶ Data hanya bisa diakses/diubah lewat getter dan setter.
- ▶ Menggunakan underscore ( `_` ) atau pagar ( `#` )

# Encapsulation dengan ( \_ )

- ▶ underscore ( \_ ) untuk menunjukkan bahwa properti tersebut bersifat private secara konvensi, meskipun secara teknis masih bisa diakses dari luar.

```
class Car{  
  constructor(brand) {  
    this._carname = brand;  
  }  
  get carname() {  
    return this._carname;  
  }  
  set carname(x) {  
    this._carname = x;  
  }  
}  
  
newcar = new Car('Pajero');  
console.log(newcar.carname); // Output: Pajero  
console.log(newcar._carname); // Output: Pajero
```

# Encapsulation dengan ( # )

- ▶ # untuk mendefinisikan private field secara benar-benar tersembunyi dari luar class.
- ▶ Properti dengan tanda # hanya bisa diakses dari dalam class.

```
class Car{  
    #carname; // <-- wajib deklarasi untuk private field  
    constructor(brand) {  
        this.#carname = brand;  
    }  
    get carname() {  
        return this.#carname;  
    }  
    set carname(x) {  
        this.#carname = x;  
    }  
}  
  
newcar = new Car('Pajero');  
console.log(newcar.carname); // Output: Pajero  
console.log(newcar.#carname); // Error : Private field
```

# Tipe Data dan Control Program dalam Class (If ... else)

- ▶ Membuat tipe data dengan nama kecepatan
- ▶ Membuat control program (If... Else) untuk menentukan perilaku mobil.

```
class Car {  
  constructor(kecepatan) {  
    this.kecepatan = kecepatan; // number  
  }  
  
  // Method: tampilkan informasi mobil  
  info() {  
    console.log(`Mobil memiliki kecepatan ${this.kecepatan} km/jam.`);  
  
    // Control program: if-else  
    if (this.kecepatan > 0) {  
      console.log("Mobil sedang berjalan.");  
    } else {  
      console.log("Mobil sedang berhenti.");  
    }  
  }  
}
```

# Tipe Data dan Control Program dalam Class (If ... else)

- ▶ Membuat objek1 dengan nama mobil1 dengan mengirimkan nilai 80.
- ▶ Membuat objek2 dengan nama mobil2 dengan mengirimkan nilai 0.

```
var mobil1 = new Car(80);  
var mobil2 = new Car(0);  
  
// Jalankan method untuk masing-masing objek  
mobil1.info(); // Output: Mobil memiliki kecepatan 80 km/jam. Mobil sedang berjalan.  
mobil2.info(); // Output: Mobil memiliki kecepatan 0 km/jam. Mobil sedang berhenti.
```

# INHERITANCE

| ELEMEN KOMPETENSI                               | KRITERIA UNJUK KERJA                                  |
|-------------------------------------------------|-------------------------------------------------------|
| 3. Membuat program dengan konsep berbasis objek | 3.1 <b>Inheritance</b> pada <b>class</b> diterapkan.  |
|                                                 | 3.2 <b>Polymorphism</b> pada <b>class</b> diterapkan. |
|                                                 | 3.3 <b>Overloading</b> pada <b>class</b> diterapkan.  |

# Definisi

- ▶ Inheritance merupakan konsep pewarisan dimana sebuah class dapat mewarisi method maupun property dari class parentnya.
- ▶ Untuk membuat inheritance dari suatu class, gunakan keyword `extends`.
- ▶ Class yang dibuat dengan metode inheritance, akan memiliki method yang sama dengan class asalnya.
- ▶ **Super class / base class / parent class** adalah class yang fiturnya diwariskan.
- ▶ **Sub class / derived class / extended class / child class** adalah Class yang mewarisi fitur dari class lain

# Inheritance Javascript

- ▶ Diawali dengan membuat *Class Induk (Parent Class)*.
- ▶ Membuat *Class Anak (Child Class)* dengan *extends*.
- ▶ Objek dapat mengakses dari *Parent Class* dan *Child Class*.

```
// Parent class
class Parent {
  constructor(greeting) {
    this.greeting = greeting;
  }

  parentsay() {
    console.log(`${this.greeting} this is the parent class.`);
  }
}

// Child class
class Child extends Parent {
  constructor(greeting, name) {
    super(greeting); // panggil constructor dari Parent
    this.name = name;
  }

  childsay() {
    console.log(`${this.greeting} , ${this.name} this is the child class.`);
  }
}
```

# Instantiate Inheritance Class

- ▶ Menampilkan method dari *parent* class dengan instantiate class, kemudian nama object dilanjutkan dengan tanda titik dan nama method di *parent class*.
- ▶ Menampilkan method dari *child* class dengan instantiate class, kemudian nama object dilanjutkan dengan tanda titik dan nama method di *child class*.

```
var parent1 = new Parent("Hai");  
parent1.parentsay(); // ✓ This is the parent class.  
  
const child1 = new Child("Hello", "Mark");  
child1.childday(); // ✓ Hello, Mark this is the child class.
```

# POLYMORPHISM

# Polymorphism (Override)

- ▶ Konsep pemrograman berbasis objek di mana objek berbeda bisa merespons method yang sama dengan cara yang berbeda.
- ▶ Satu nama method, banyak perilaku tergantung dari objek yang memanggil.
- ▶ Membuat kode lebih fleksibel dan mudah diperluas

# Polymorphism (Override)

- ▶ *Parent Class* dan *Child Class* menggunakan nama method yang sama yaitu `sound` namun memiliki perilaku yang berbeda.

```
class Vehicle {  
    sound() {  
        console.log("This vehicle makes a sound.");  
    }  
}  
  
class Car extends Vehicle {  
    sound() {  
        console.log("Vroom vroom!");  
    }  
}  
  
class Bike extends Vehicle {  
    sound() {  
        console.log("Ring ring!");  
    }  
}
```

# Instance Polymorphism (Override)

- ▶ Instantiate objek akan mempengaruhi perilaku dari objek yang dibuat.
- ▶ Objek akan mengikuti perilaku dari class yang dipanggil saat instantiate.

```
// Instance Polymorphism
var vehicle1 = new Vehicle();
vehicle1.sound(); // Output: This vehicle makes a sound.
var vehicle2 = new Car();
vehicle2.sound(); // Output: Vroom vroom!
var vehicle3 = new Bike();
vehicle3.sound(); // Output: Ring ring!
```

# Polymorphism (Overload)

- ▶ Satu nama method, tapi parameter berbeda-beda jumlah atau tipe.
- ▶ JavaScript tidak punya method overloading bawaan, tapi bisa mensimulasikannya dengan logika : `arguments.length`, `typeof`, atau `default values`.

# Polymorphism (Overload)

- ▶ Tipedata divalidasi menggunakan typeof dengan control program.
- ▶ Hasil validasi akan ditampilkan oleh method.

```
class TypeNumber {  
  constructor(number) {  
    this.number = number;  
    if (typeof number === "string") {  
      this.keterangan = `Ini bukan angka, tapi string: ${number}`;  
    } else {  
      this.keterangan = `Ini adalah angka: ${number}`;  
    }  
  }  
  
  info() {  
    console.log(this.keterangan);  
  }  
}
```

# Instance Polymorphism (Overload)

- ▶ Tipe data divalidasi menggunakan typeof dengan control program.
- ▶ Hasil validasi akan ditampilkan oleh method.

```
var number1 = new TypeNumber(20);  
number1.info(); // Ini adalah angka: 20  
  
var number2 = new TypeNumber("20");  
number2.info(); // Ini bukan angka, tapi string: 20
```

| ELEMEN KOMPETENSI                                                                         | KRITERIA UNJUK KERJA                                                                         |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 4. Membuat program <i>object oriented</i> dengan <b><i>interface</i></b> dan <b>paket</b> | 4.1 <b><i>Interface class</i></b> program dibuat.<br>4.2 <b>Paket</b> dengan program dibuat. |

# INTERFACE

# Interface

- ▶ Template yang mendefinisikan metode dan properti yang wajib dimiliki oleh suatu class, tanpa memberikan implementasinya.
- ▶ Jika sebuah class mengimplementasikan interface, maka class tersebut harus memenuhi semua aturan yang ditetapkan oleh interface tersebut.
- ▶ JavaScript tidak punya interface secara bawaan, namun dapat meniru konsep interface dengan teknik tertentu.

# Interface dengan Typescript

- ▶ File dibuat dengan extension *namafile.ts* karena menggunakan typescript.
- ▶ Tambahkan kata interface pada *parent class*.
- ▶ Pada *child class* tambahkan *implements* dilanjutkan dengan *interface 1, interface 2*

```
// Interface Animal {
interface Animal {
  nama: string;
  suara(): void;
}

// Interface category
interface Category {
  deskripsi: string;
}

// Class Cat implements Animal and Category interfaces
class Cat1 implements Animal, Category {
  nama: string;
  deskripsi: string;

  constructor(nama: string, deskripsi: string) {
    this.nama = nama;
    this.deskripsi = deskripsi;
  }

  suara(): void {
    console.log(`${this.nama} ${this.deskripsi} says Meow!`);
  }
}

// Instantiate the Cat class
const kucing1 = new Cat("Kitty", "Mamalia"); // Create an instance of Cat
kucing1.suara(); // Output: Kitty Mamalia says Meow!
```

# Interface dengan Typescript

- ▶ Install TypeScript secara global dengan perintah:  
**npm install -g typescript**
- ▶ Compile ke javascript dengan perintah:  
**tsc namafile.ts**
- ▶ Ini akan menghasilkan file namafile.js di folder yang sama.
- ▶ Jalankan file JavaScript-nya dengan perintah :  
**node namafile.js**

# PACKAGE

# Definisi

- ▶ Kumpulan kode (fungsi, modul, atau library) yang dapat digunakan kembali dalam proyek JavaScript.

# Membuat package

- ▶ Setelah class dibuat, ketikkan `module.exports = namaclass`

```
class Car {  
  constructor(merek) {  
    this.merek = merek;  
  }  
  
  info() {  
    console.log(`Car brand : ${this.merek}.`);  
  }  
}  
  
module.exports = Car;
```

# Menggunakan package

- ▶ Import class dengan membuat variable diikuti perintah `require(' ./lokasi file dari class');`

```
const Car = require('./vehicle'); // import class Car dari package lokal

const mobil1 = new Car("Mitsubishi");
mobil1.info(); // Output: Car brand : Mitsubishi.
```

| ELEMEN KOMPETENSI        | KRITERIA UNJUK KERJA                                                          |
|--------------------------|-------------------------------------------------------------------------------|
| 5. Mengkompilasi Program | 5.1 Kesalahan dapat dikoreksi.<br>5.2 Program bebas salah sintaks dihasilkan. |

# TRY CATCH

# Kompilasi dengan try catch

Menampilkan pesan error jika nilai yang dimasukkan tidak sesuai

```
class Car {
  constructor(kecepatan) {
    this.kecepatan = kecepatan;
  }

  tambahKecepatan(nilai) {
    try {
      if (typeof nilai !== "number") {
        throw new Error("Kecepatan harus berupa angka!");
      }

      console.log(`Kecepatan sekarang ${this.kecepatan} km/jam`);
    } catch (error) {
      console.log(`Terjadi kesalahan: ${error.message}`);
    }
  }
}

// Menggunakan class Car
const mobil = new Car(50);
mobil.tambahKecepatan(0); // Output: Kecepatan sekarang 50 km/jam
//mobil.tambahKecepatan("cepat"); // Output: Terjadi kesalahan: Kecepatan harus berupa angka!
```

# Aplikasi

- ▶ Apa yang Anda ketahui tentang Pemrograman berbasis objek?
- ▶ Sebutkan karakteristik dari Pemrograman Berbasis Objek

# Evaluasi

- ▶ Kerjakan soal berikut dengan memilih jawaban yang paling benar.

# Tugas Tambahan

- ▶ Buatlah tugas sesuai dengan instruksi yang telah dibuat.



KEMENTERIAN  
KETENAGAKERJAAN  
REPUBLIK INDONESIA

# SEKIAN DAN TERIMA KASIH

Sampai jumpa di pertemuan selanjutnya

**Salam Kompeten !**



2025

Smart **Creative Skills** Team ©